CSE563 — Unit VI Numerical Worksheet

Algorithmic and Optimization Approaches

40 questions • CO6 • Solutions Booklet

Instructor Note

Full worked solutions for all 40 questions. Every dynamic-programming answer includes the complete table *and* the traceback; every backtracking answer shows the pruned branches.

A. Greedy Approach (Q1–Q8)

Q1. Activities with (start, finish): $(1,4), (3,5), (0,6), (5,7), (3,9), (5,9), (6,10), (8,11), (8,12), (2,14), (12,16)$. Apply the greedy rule and give the maximum set of non-overlapping activities.

Solution. Sort by finish time (already sorted above). Select the earliest-finishing compatible activity each time.

Activity	Start $\geq$ last finish?	Decision
$(1,4)$	—	**select**; last $= 4$
$(3,5)$	$3 < 4$	reject
$(0,6)$	$0 < 4$	reject
$(5,7)$	$5 \geq 4$	**select**; last $= 7$
$(3,9)$, $(5,9)$, $(6,10)$	< 7	reject
$(8,11)$	$8 \geq 7$	**select**; last $= 11$
$(8,12)$	$8 < 11$	reject
$(2,14)$	$2 < 11$	reject
$(12,16)$	$12 \geq 11$	**select**

Answer: 4 activities — $(1,4), (5,7), (8,11), (12,16)$**.**

Q2. State the greedy-choice property and optimal substructure, and explain why sorting by *finish* time (rather than start time) is the correct rule for activity selection.

Solution.

Greedy-choice property — a locally optimal choice is contained in *some* globally optimal solution, so committing to it never rules out optimality.

Optimal substructure — an optimal solution to the whole problem contains optimal solutions to its subproblems.

Why earliest finish. Choosing the activity that finishes earliest leaves the *maximum possible remaining time* for everything that follows, so it can never be worse than any other first choice. *Exchange argument:* let O be any optimal solution and a_1 the earliest-finishing activity overall. If O does not contain a_1, replace O's first activity with a_1. Since a_1 finishes no later, the rest of O remains compatible, and $|O|$ is unchanged. So an optimal solution containing a_1 exists. ■

Why earliest start fails: a very long activity may start first and block many short ones — e.g. $(0,6)$ blocks both $(1,4)$ and $(5,7)$.

Q3. Fractional knapsack with capacity $W = 20$ and items (w, v): $A(5,50)$, $B(10,60)$, $C(8,64)$, $D(4,20)$. Find the maximum value.

Solution.

Item	w	v	v/w	Rank
A	5	50	**10.0**	1
C	8	64	8.0	2
B	10	60	6.0	3
D	4	20	5.0	4

Take all of A (5, value 50); all of C (8, total weight 13, value 64); then B — only $20 - 13 = 7$ of its 10 units fit, giving $\frac{7}{10}(60) = 42$.

$$\text{Total} = 50 + 64 + 42 = \mathbf{176}, \quad \text{weight exactly 20.}$$

Q4. Repeat the previous question as a *0/1* knapsack (items cannot be divided) and compare with the greedy answer.

Solution. **Greedy by ratio (0/1):** take A (5, 50), then C (8, total 13, value 114). B needs 10 but only 7 remain — skip. D needs 4, and 7 remain ✓: weight 17, value 134. **Greedy gives 134.**
True optimum: try $A + C + D$ = weight 17, value 134. Try $A + B + D$ = weight 19, value 130. Try $C + B$ = weight 18, value 124. Try $A + C + B$ = weight 23 — exceeds. Try $A + C + D$ again, 17/134. The best achievable is **134** with $\{A, C, D\}$, weight 17.
Comparison. Here greedy happens to find the optimum, but that is luck, not a guarantee — three units of capacity go unused. The fractional answer 176 is strictly larger, and the gap $(176 - 134 = 42)$ shows that the fractional relaxation is only an *upper bound* on the 0/1 optimum. For a guaranteed 0/1 answer, dynamic programming is required.

Q5. Construct Huffman codes for frequencies A:45, B:13, C:12, D:16, E:9, F:5. Give the code lengths, the total bits and the saving over a fixed-length code.

Solution. **Merging (always the two smallest):**

Step	Merge	New node
1	$F(5) + E(9)$	14
2	$C(12) + B(13)$	25
3	$14 + D(16)$	30
4	$25 + 30$	55
5	$A(45) + 55$	100 (root)

Codes: $A = 0$ (1 bit); $C = 100$, $B = 101$, $D = 111$ (3 bits); $F = 1100$, $E = 1101$ (4 bits).

$$\text{Total} = 45(1) + 13(3) + 12(3) + 16(3) + 9(4) + 5(4) = 45 + 39 + 36 + 48 + 36 + 20 = \mathbf{224} \text{ bits.}$$

Average = 2.24 bits/symbol. **Fixed-length:** 6 symbols need $\lceil \log_2 6 \rceil = 3$ bits, i.e. 300 bits. **Saving** $= 300 - 224 = 76$ **bits, i.e.** 25.3%**.** *Prefix check:* $0, 100, 101, 111, 1100, 1101$ — none is a prefix of another ✓.

Q6. For coins $\{1, 7, 10\}$ and amount 15, show that the greedy algorithm is not optimal.

Solution. **Greedy (largest first):** 10, then $15 - 10 = 5$ needs five 1s.

$$10 + 1 + 1 + 1 + 1 + 1 = \mathbf{6 \text{ coins}}.$$

Optimal: $7 + 7 + 1 = \mathbf{3 \text{ coins}}$.

Greedy uses twice as many coins. The system $\{1, 7, 10\}$ is **not canonical**: committing to the 10 leaves a remainder that the other denominations cannot cover efficiently. The greedy-choice property fails, so dynamic programming must be used.

Q7. Give one problem where greedy is provably optimal and one where it is not, stating in each case which property holds or fails.

Solution. **Provably optimal — Minimum Spanning Tree (Kruskal/Prim).** The *cut property* guarantees that the lightest edge crossing any cut belongs to some MST, so the greedy choice is always safe. Both greedy-choice and optimal substructure hold.
Not optimal — 0/1 knapsack. Greedy by value/weight ratio can be arbitrarily bad. With capacity 10 and items $(6, 30), (5, 25), (5, 25)$ (all ratio 5), greedy takes the 6-weight item and reaches value 30, whereas the two 5-weight items give 50. The *greedy-choice property fails* because an item cannot be split to use the leftover capacity.
Other optimal cases: activity selection, Huffman coding, fractional knapsack, Dijkstra.

Q8. A machine processes jobs with durations $6, 3, 8, 2, 5$. Scheduling to minimise the *average completion time* is greedy. Give the rule, the schedule and the average.

Solution. **Rule: Shortest Job First (SJF).** Sorting ascending minimises the sum of completion times, because a short job placed early reduces the waiting time of every job after it. Sorted: $2, 3, 5, 6, 8$.

Job duration	Completion time	Running total
2	2	2
3	5	7
5	10	17
6	16	33
8	24	57

Average completion time $= 57/5 = \mathbf{11.4}$.
Compare with the worst order (longest first: $8, 6, 5, 3, 2$): completions $8, 14, 19, 22, 24$, total 87, average 17.4. SJF is markedly better.
Exchange argument: if a longer job precedes a shorter one, swapping them reduces the total completion time, so any optimal schedule must be in non-decreasing order of duration.

B. Divide and Conquer (Q9–Q14)

Q9. Multiply $x = 5678$ and $y = 1234$ using Karatsuba's method, showing the three products.

Solution. Split: $a = 56$, $b = 78$, $c = 12$, $d = 34$.

$$P_1 = ac = 56 \times 12 = 672$$
$$P_2 = bd = 78 \times 34 = 2652$$
$$P_3 = (a + b)(c + d) = 134 \times 46 = 6164$$
$$\text{middle} = P_3 - P_1 - P_2 = 6164 - 672 - 2652 = 2840$$

$$xy = 672 \times 10^4 + 2840 \times 10^2 + 2652 = 6{,}720{,}000 + 284{,}000 + 2{,}652 = \mathbf{7{,}006{,}652}.$$

Check: $5678 \times 1234 = 7{,}006{,}652$ ✓. Only **three** half-size multiplications were needed instead of four.

Q10. State the recurrences for schoolbook and Karatsuba multiplication and solve both, quantifying the improvement.

Solution. **Schoolbook:** four half-size products plus linear combination work:

$$T(n) = 4T(n/2) + n. \quad d = \log_2 4 = 2, \ f = n = O(n^{2-1}) \Rightarrow \text{Case 1} \Rightarrow \mathbf{\Theta(n^2)}.$$

Karatsuba: three half-size products:

$$T(n) = 3T(n/2) + n. \quad d = \log_2 3 \approx 1.585, \ f = n = O(n^{1.585-\varepsilon}) \Rightarrow \text{Case 1} \Rightarrow \mathbf{\Theta(n^{1.585})}.$$

Improvement. For $n = 1024$ digits: $n^2 \approx 1.05\times10^6$ versus $n^{1.585} \approx 5.9\times10^4$ digit-multiplications — roughly an 18-fold saving, and the gap widens with n.

Q11. Find the maximum subarray sum of $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$ using the divide-and-conquer approach. Show the three cases.

Solution. Indices 0–8; split at mid = 4: left = $A[0..4] = [-2, 1, -3, 4, -1]$, right = $A[5..8] = [2, 1, -5, 4]$.
Crossing. Best suffix of the left, ending at index 4 — running sums leftward from $A[4] = -1$: $-1, 3, 0, 1, -1$; maximum = **3** (subarray $[4, -1]$). Best prefix of the right, starting at index 5: $2, 3, -2, 2$; maximum = **3** (subarray $[2, 1]$). Crossing sum = $3 + 3 =$ **6**, subarray $A[3..6] = [4, -1, 2, 1]$.
Purely left: best is [4] with sum 4. **Purely right:** best is $[2, 1]$ with sum 3, or [4] with 4 — maximum 4.
Overall maximum = 6**,** achieved by $[4, -1, 2, 1]$. *Kadane's $O(n)$ check:* best-ending-here $= -2, 1, -2, 4, 3, 5, 6, 1, 5$; maximum = 6 ✓.

Q12. Write the recurrence for the divide-and-conquer maximum-subarray algorithm, solve it, and compare with Kadane's algorithm.

Solution. Two halves plus a linear scan for the crossing case:

$$T(n) = 2T(n/2) + \Theta(n).$$

Master theorem: $a = 2$, $b = 2$, $d = 1$, $f = n = \Theta(n^1)$ — **Case 2**:

$$T(n) = \mathbf{\Theta(n \log n)}.$$

Kadane's algorithm is a one-pass dynamic program: keep `cur = max(x, cur+x)` and `best = max(best, cur)`, running in $\mathbf{\Theta(n)}$ time and $\Theta(1)$ space.
Kadane is strictly better here. The divide-and-conquer version is retained for teaching because it illustrates the "combine" step costing $\Theta(n)$, and because it generalises to settings (such as 2-D) where a linear scan does not.

Q13. State and solve the recurrences for merge sort, binary search and Strassen's matrix multiplication, naming the master-theorem case in each.

Solution.

Algorithm	Recurrence	$d = \log_b a$	Case	Result
Merge sort	$2T(n/2) + n$	1	2	$\Theta(n \log n)$
Binary search	$T(n/2) + 1$	0	2	$\Theta(\log n)$
Strassen	$7T(n/2) + n^2$	$\log_2 7 \approx 2.807$	1	$\Theta(n^{2.807})$

Merge sort: $f = n = \Theta(n^1)$, equal to n^d — Case 2. **Binary search:** $n^d = n^0 = 1$ and $f = 1 = \Theta(1)$ — Case 2. **Strassen:** $f = n^2 = O(n^{2.807-0.8})$, polynomially smaller — Case 1, so the leaves dominate. Naive multiplication is $\Theta(n^3)$, so Strassen is asymptotically faster.

Q14. Explain the difference between divide-and-conquer and dynamic programming, and state the property that decides which to use.

Solution. Both rely on **optimal substructure** and both break a problem into subproblems. The deciding property is whether the subproblems **overlap**.

Divide and conquer — the subproblems are *disjoint* — merge sort's two halves never share an element — so each is solved exactly once and plain recursion is efficient.

Dynamic programming — the subproblems *recur*, so naive recursion re-solves them exponentially many times. DP stores each result once (memoisation top-down, or tabulation bottom-up) and reuses it.

Canonical illustration. Naive recursive Fibonacci is $\Theta(\varphi^n)$ because `fib(n-2)` is computed twice, `fib(n-3)` three times, and so on. Memoising collapses this to $\Theta(n)$ — the same recurrence, but each of the n subproblems solved once.

C. Dynamic Programming (Q15–Q26)

Q15. Solve the 0/1 knapsack with capacity $W = 7$ and items (w, v): 1(1, 1), 2(3, 4), 3(4, 5), 4(5, 7). Give the full table.

Solution. $\text{dp}[i][c] = \max(\text{dp}[i-1][c],\ \text{dp}[i-1][c-w_i] + v_i)$ when $w_i \le c$.

$i \backslash c$	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1 (1, 1)	0	1	1	1	1	1	1	1
2 (3, 4)	0	1	1	4	5	5	5	5
3 (4, 5)	0	1	1	4	5	6	6	**9**
4 (5, 7)	0	1	1	4	5	7	8	**9**

Sample: $\text{dp}[3][7] = \max(\text{dp}[2][7],\ \text{dp}[2][3] + 5) = \max(5, 4 + 5) = 9$. **Optimal value** $= 9$**.**

Q16. Trace back through the table of the previous question to identify which items are chosen, and verify the weight and value.

Solution.

Cell	Comparison	Conclusion
$\text{dp}[4][7] = 9$	$= \text{dp}[3][7] = 9$	item 4 **not** taken
$\text{dp}[3][7] = 9$	$\neq \text{dp}[2][7] = 5$	item 3 **taken**; $c \to 3$
$\text{dp}[2][3] = 4$	$\neq \text{dp}[1][3] = 1$	item 2 **taken**; $c \to 0$
$\text{dp}[1][0] = 0$	$= \text{dp}[0][0] = 0$	item 1 not taken

Items chosen: 2 and 3. Weight $= 3 + 4 = 7 \le 7$ ✓; value $= 4 + 5 = 9$ ✓. *Greedy comparison:* ratios are $1.0, 1.33, 1.25, 1.4$, so greedy takes item 4 (value 7), then item 1 fits (value 8). Greedy

gives 8 — worse than the optimal 9.

Q17. Solve the 0/1 knapsack with $W = 8$ and items (w, v): $1(3,4)$, $2(4,5)$, $3(2,3)$. Give the table, the optimal value and the items.

Solution.

$i\backslash c$	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1 $(3,4)$	0	0	0	4	4	4	4	4	4
2 $(4,5)$	0	0	0	4	5	5	5	9	9
3 $(2,3)$	0	0	3	4	5	7	8	9	**9**

Sample cells. dp[2][7] = max(dp[1][7], dp[1][3] + 5) = max(4, 4 + 5) = 9. dp[3][5] = max(dp[2][5], dp[2][3] + 3) = max(5, 4 + 3) = 7. dp[3][8] = max(dp[2][8], dp[2][6] + 3) = max(9, 5 + 3) = **9**.
Optimal value = 9**.**
Traceback: dp[3][8] = 9 = dp[2][8], so item 3 is *not* taken. dp[2][8] = 9 ≠ dp[1][8] = 4, so item 2 **is** taken, leaving $c = 4$. dp[1][4] = 4 ≠ dp[0][4] = 0, so item 1 **is** taken. **Items 1 and 2**, weight $3 + 4 = 7 \le 8$, value $4 + 5 = 9$ ✓.
Why item 3 is excluded: all three would weigh $3 + 4 + 2 = 9 > 8$. The best alternatives are items 1+3 (weight 5, value 7) and items 2+3 (weight 6, value 8) — both worse than 9.

Q18. Find the LCS of $X =$ `ABCD` and $Y =$ `ACBAD`, giving the table and the traceback.

Solution.

	∅	A	C	B	A	D
∅	0	0	0	0	0	0
A	0	1	1	1	1	1
B	0	1	1	2	2	2
C	0	1	2	2	2	2
D	0	1	2	2	2	**3**

LCS length = 3**. Traceback** from $L[4][5]$: $X_4 = \text{D} = Y_5 = \text{D} \Rightarrow$ record **D**, go to $L[3][4]$. $X_3 = \text{C}$, $Y_4 = \text{A}$: $L[2][4] = 2 = L[3][3] = 2$ — tie; move left to $L[3][3]$. $X_3 = \text{C}$, $Y_3 = \text{B}$: $L[2][3] = 2 > L[3][2] = 2$ — tie; move up to $L[2][3]$. $X_2 = \text{B} = Y_3 = \text{B} \Rightarrow$ record **B**, go to $L[1][2]$. $X_1 = \text{A}$, $Y_2 = \text{C}$: move left to $L[1][1]$. $X_1 = \text{A} = Y_1 = \text{A} \Rightarrow$ record **A**. Reversed: **LCS** = `ABD`. (`ACD` is an equally valid length-3 answer.)

Q19. Find the LCS length of $X =$ `AGGTAB` and $Y =$ `GXTXAYB`.

Solution.

	∅	G	X	T	X	A	Y	B
∅	0	0	0	0	0	0	0	0
A	0	0	0	0	0	1	1	1
G	0	1	1	1	1	1	1	1
G	0	1	1	1	1	1	1	1
T	0	1	1	2	2	2	2	2
A	0	1	1	2	2	3	3	3
B	0	1	1	2	2	3	3	**4**

LCS length = 4, and the traceback gives `GTAB`. *Check:* `GTAB` appears in A<u>G</u>G<u>TAB</u> and in <u>G</u>X<u>T</u>X<u>A</u>Y<u>B</u> ✓.

Q20. Solve the minimum coin change for coins $\{1, 3, 4\}$ and amount 6, giving the full DP array, and compare with the greedy answer.

Solution. $\mathrm{dp}[0] = 0$; $\mathrm{dp}[a] = 1 + \min_{c \le a} \mathrm{dp}[a - c]$.

a	0	1	2	3	4	5	6
$\mathrm{dp}[a]$	0	1	2	1	1	2	**2**
coin used	–	1	1	3	4	4	3

Sample: $\mathrm{dp}[6] = 1 + \min(\mathrm{dp}[5], \mathrm{dp}[3], \mathrm{dp}[2]) = 1 + \min(2, 1, 2) = \mathbf{2}$. **Minimum** $= 2$ **coins** $(3 + 3)$. **Greedy:** largest first gives $4 + 1 + 1 = \mathbf{3}$ **coins** — worse. The system $\{1, 3, 4\}$ is not canonical.

Q21. Solve the minimum coin change for coins $\{2, 5, 10\}$ and amount 13, or show it is impossible.

Solution.

a	0	1	2	3	4	5	6	7	8	9	10	11	12	13
$\mathrm{dp}[a]$	0	∞	1	∞	2	1	3	2	4	3	1	4	2	**5**

Sample computations. $\mathrm{dp}[1] = \infty$ — no coin is small enough, and odd amounts below 5 are unreachable. $\mathrm{dp}[9] = 1 + \min(\mathrm{dp}[7],\ \mathrm{dp}[4]) = 1 + \min(2, 2) = 3$. $\mathrm{dp}[13] = 1 + \min(\mathrm{dp}[11],\ \mathrm{dp}[8],\ \mathrm{dp}[3]) = 1 + \min(4,\ 4,\ \infty) = \mathbf{5}$.
Minimum $= 5$ **coins:** $13 = 5 + 2 + 2 + 2 + 2$.
Parity argument confirming this is optimal. Every coin except the 5 is even, and 13 is odd, so an *odd* number of 5s must be used. Using one 5 leaves 8, which needs four 2s (the 10 is too large) — total 5 coins. Using three 5s leaves -2, impossible. Hence 5 coins is the minimum, and the amount *is* reachable.
Note: amounts 1 and 3 are unreachable altogether ($\mathrm{dp} = \infty$), which is why a solution must always check for ∞ before reporting an answer.

Q22. Compute the Fibonacci number F_{10} by tabulation, and state the complexity of the naive recursive method versus the DP method.

Solution.

n	0	1	2	3	4	5	6	7	8	9	10
F_n	0	1	1	2	3	5	8	13	21	34	**55**

$F_{10} = 55$. **Naive recursion:** $T(n) = T(n-1) + T(n-2) + 1$, whose solution grows as $\Theta(\varphi^n)$ with $\varphi \approx 1.618$ — **exponential**, because `fib(n-2)` is recomputed twice, `fib(n-3)` three times, and so on. **DP (memoisation or tabulation):** $n + 1$ subproblems each solved once in $O(1)$, so $\Theta(n)$ time; with two rolling variables, $\Theta(1)$ space.

Q23. State the five steps of designing a dynamic-programming solution, and apply them to the rod-cutting problem informally.

Solution. **The five steps.**

1. **Define the state** — what does $\mathrm{dp}[i]$ (or $\mathrm{dp}[i][j]$) mean?
2. **Write the recurrence** relating a state to smaller states.
3. **Set the base cases.**
4. **Choose an evaluation order** — memoisation (top-down) or tabulation (bottom-up).
5. **Reconstruct the solution** by tracing which choice produced each value.

Applied to rod cutting (a rod of length n, price p_i for a piece of length i; maximise revenue):

1. State: $\mathrm{dp}[j]$ = maximum revenue obtainable from a rod of length j.
2. Recurrence: $\mathrm{dp}[j] = \max_{1 \le i \le j} (p_i + \mathrm{dp}[j-i])$ — try every length for the first cut.
3. Base: $\mathrm{dp}[0] = 0$.
4. Order: compute $j = 1, 2, \ldots, n$ in increasing order.
5. Reconstruct: store the i achieving each maximum and follow it back.

Complexity: $\Theta(n^2)$ — n states, each taking $O(n)$ work.

Q24. Explain the difference between memoisation and tabulation, with the advantages of each.

Solution.
Memoisation (top-down) — write the natural recursion and add a cache; before computing a subproblem, check whether its answer is already stored.
Tabulation (bottom-up) — fill a table iteratively from the base cases upward, in an order guaranteeing that every dependency is already computed.

	Memoisation	**Tabulation**
Code shape	closest to the recurrence, easiest to write	requires working out a valid order
Subproblems solved	*only those reachable* from the input	all of them
Overhead	recursion-stack and function-call cost	none; plain loops
Risk	stack overflow on deep recursion	may compute unnecessary states
Space saving	harder	easy — often only the previous row is needed

Both give the same asymptotic time. Choose memoisation when many subproblems are unreachable; choose tabulation when all are needed or when space must be optimised.

Q25. Name four classic dynamic-programming problems and give the state and recurrence for each.

Solution.

Problem	State	Recurrence
0/1 knapsack	$\mathrm{dp}[i][c]$: best value from first i items, capacity c	$\max(\mathrm{dp}[i-1][c],\ \mathrm{dp}[i-1][c-w_i] + v_i)$
LCS	$L[i][j]$: LCS of $X[1..i]$, $Y[1..j]$	match $\Rightarrow L[i-1][j-1]+1$; else $\max(L[i-1][j], L[i][j-1])$
Coin change	$\mathrm{dp}[a]$: fewest coins for amount a	$1 + \min_{c \le a} \mathrm{dp}[a-c]$
Floyd–Warshall	$d^{(k)}[i][j]$: shortest $i \to j$ using intermediates $\le k$	$\min\big(d^{(k-1)}[i][j],\ d^{(k-1)}[i][k] + d^{(k-1)}[k][j]\big)$

Others worth naming: edit distance, longest increasing subsequence, matrix-chain multiplication, rod cutting, Bellman–Ford, Viterbi.

Q26. Explain why the 0/1 knapsack DP running in $\Theta(nW)$ does not prove that knapsack is in P.

Solution. The algorithm is **pseudo-polynomial**: it is polynomial in the *numeric value* W, not in the *length of the input encoding*.
An input is written in binary, so the capacity W occupies only $\log_2 W$ bits. Writing $L = \log_2 W$ for that length, the running time is

$$\Theta(nW) = \Theta(n\, 2^L),$$

which is **exponential** in the input size L. For example, a single capacity of $W = 2^{60}$ takes 60

bits to write down but would require about 10^{18} table cells.
A genuine polynomial-time algorithm must be polynomial in the *encoded* input length. Hence 0/1 knapsack remains NP-complete despite the table method, and the DP is practical only when W is small.

D. Backtracking (Q27–Q32)

Q27. State the two conditions checked when placing a queen in the N-Queens problem, and explain why no row check is needed.

Solution. Represent a partial solution as a list of column indices, one entry per row. Before placing a queen at column c in row r, compare with every earlier queen (r', c'):

1. **Same column:** reject if $c = c'$.
2. **Same diagonal:** reject if $|r - r'| = |c - c'|$.

No row check is needed because the representation places *exactly one queen per row by construction* — the r-th entry of the list *is* the queen in row r, so two queens can never share a row. This encoding choice eliminates one entire class of constraint and shrinks the search space from $\binom{N^2}{N}$ placements to N^N (and to $N!$ if a column check is also built in).

Q28. Trace the backtracking search for 4-Queens starting from column 0 in row 0, showing where pruning occurs.

Solution.

Partial	Outcome
$[0]$	row 1: col 0 same column ×; col 1 diagonal ($\|1-0\| = \|1-0\|$) ×; col 2 valid
$[0, 2]$	row 2: col 0 column clash with $(0, 0)$ ×; col 1 diagonal from $(1, 2)$ ×; col 2 column ×; col 3 diagonal from $(1, 2)$ × ⇒ **PRUNE**
$[0, 3]$	row 1 retried with col 3: valid. row 2: col 1 valid
$[0, 3, 1]$	row 3: col 0 × (column, from $(0, 0)$); col 1 ×; col 2 × (diagonal from $(2, 1)$); col 3 × ⇒ **PRUNE** row 2 has no further columns ⇒ backtrack; row 1 exhausted ⇒ backtrack
$[0]$	all continuations exhausted — **no solution begins with column 0**

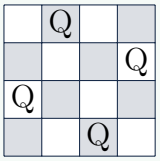
The search then tries $[1]$, which leads to the solution $[1, 3, 0, 2]$.
Pruning saving: only about six partial boards were examined in the $[0]$ subtree, whereas brute force would have tested all $4^3 = 64$ completions of it.

Q29. Give both solutions to the 4-Queens problem and state the relationship between them.

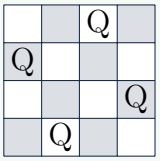
Solution.

$$[1, 3, 0, 2] \qquad \text{and} \qquad [2, 0, 3, 1]$$

(reading as: the queen in row r sits in column given by entry r).



$[1, 3, 0, 2]$



$[2, 0, 3, 1]$

Relationship: the two are **mirror images** of each other in a vertical axis — applying $c \mapsto 3 - c$ to each entry of $[1, 3, 0, 2]$ gives $[2, 0, 3, 1]$. For $N = 8$ there are 92 solutions, which reduce to 12 distinct ones under the symmetries of the square.

Q30. Find all subsets of $\{2, 4, 6, 8\}$ summing to 10 by backtracking, showing the pruning.

Solution. Sort ascending; total available = 20. Prune when the running sum exceeds 10.

Partial (sum)	Action
$\{2\}$ (2)	continue
$\{2, 4\}$ (6)	continue
$\{2, 4, 6\}$ (12)	$12 > 10 \Rightarrow$ **prune**
$\{2, 4, 8\}$ (14)	$> 10 \Rightarrow$ **prune**
$\{2, 6\}$ (8)	continue; $\{2, 6, 8\}$ (16) prune; 8 alone $\neq 10$
$\{2, 8\}$ (10)	**SOLUTION**
$\{4\}$ (4)	continue
$\{4, 6\}$ (10)	**SOLUTION**
$\{4, 8\}$ (12)	$> 10 \Rightarrow$ prune
$\{6\}$ (6)	$\{6, 8\}$ (14) prune; 6 alone $\neq 10$
$\{8\}$ (8)	nothing left to add; $\neq 10$

Two solutions: $\{2, 8\}$ and $\{4, 6\}$. Brute force would examine all $2^4 = 16$ subsets; pruning cut the search to about 11 states.

Q31. Explain the difference between backtracking and branch and bound, naming a problem suited to each.

Solution.

Backtracking — a depth-first search over *partial* solutions. Extend one decision at a time; as soon as a **constraint** is violated, abandon the branch and undo the last decision. It prunes on **infeasibility**.

Branch and bound — the optimisation variant. In addition to infeasibility, it computes a **bound** on the best value attainable within a branch and prunes any branch whose bound cannot beat the best complete solution found so far (the *incumbent*). It prunes on **sub-optimality** as well.

Suited to backtracking: N-Queens, Sudoku, graph colouring, SAT via DPLL — *feasibility* problems where any valid solution will do.

Suited to branch and bound: the travelling salesman problem, 0/1 knapsack for large instances, integer linear programming — *optimisation* problems where a bound (e.g. the fractional-knapsack relaxation, or an MST-based lower bound for TSP) is available.

Q32. Name three problems solved by backtracking and describe the pruning condition used in each.

Solution.

Problem	Pruning condition
N-Queens	a proposed queen shares a column or a diagonal with an already-placed queen
Sudoku	the candidate digit already appears in that row, column or 3×3 box
Graph colouring	the candidate colour already appears on an adjacent vertex
Subset sum	the running sum exceeds the target, or (running sum + all remaining) falls below it
SAT (DPLL)	a clause becomes empty under the current partial assignment (a conflict)

In every case the pruning test is *cheap* relative to exploring the subtree it eliminates — which is precisely what makes backtracking practical despite its exponential worst case.

E. Genetic and Nature-Inspired Algorithms (Q33–Q40)

Q33. A GA maximises $f(x) = x^2$ on $x \in [0, 31]$ with 5-bit chromosomes $P_1 = 01000$, $P_2 = 11011$, $P_3 = 00111$, $P_4 = 10100$. Decode each and compute the fitness, selection probability and expected copies.

Solution.

	Chromosome	x	$f = x^2$	$p_i = f_i / \sum f$	Expected copies (N=4)
P_1	01000	8	64	0.0515	0.21
P_2	11011	27	**729**	**0.5870**	**2.35**
P_3	00111	7	49	0.0395	0.16
P_4	10100	20	400	0.3221	1.29
			$\sum = 1242$	1.0000	4.01

Decoding: $01000_2 = 8$, $11011_2 = 16 + 8 + 2 + 1 = 27$, $00111_2 = 7$, $10100_2 = 16 + 4 = 20$.
Average fitness $= 1242/4 =$ **310.5**; best = **729**. P_2 will dominate the mating pool; P_1 and P_3 are likely eliminated.

Q34. Using the mating pool $\{P_2, P_2, P_4, P_1\}$ from the previous question, perform single-point crossover on (P_2, P_4) at position 2 and on (P_2, P_1) at position 3. Decode the four children.

Solution. $P_1 = 01000$, $P_2 = 11011$, $P_4 = 10100$.
Pair (P_2, P_4), cut after bit 2: 11|011 and 10|100

$$C_1 = 11100 = \mathbf{28}\ (f = 784), \qquad C_2 = 10011 = \mathbf{19}\ (f = 361).$$

Pair (P_2, P_1), cut after bit 3: 110|11 and 010|00

$$C_3 = 11000 = \mathbf{24}\ (f = 576), \qquad C_4 = 01011 = \mathbf{11}\ (f = 121).$$

New population fitness: $784 + 361 + 576 + 121 = 1842$; average = **460.5** (up from 310.5); best = **784** (up from 729). **Both the average and the best improved.**

Q35. Apply a mutation flipping the second bit of $C_1 = 11100$ from the previous question. Give the new value and fitness, and comment on whether the mutation was beneficial.

Solution. $C_1 = 11100$; flipping bit 2 gives $10100 =$ **20**, with $f =$ **400**.
The fitness fell from 784 to 400, so this mutation was **harmful** — it destroyed the best individual in the population.
Comment. Mutation is a random operator and is as likely to harm as to help; its purpose is not immediate improvement but the maintenance of *diversity*, reintroducing alleles that selection has eliminated. That is why:

- mutation rates are kept low ($p_m \approx 0.001$–0.05 per gene), and
- **elitism** is normally used — the best individual is copied through unchanged, so a harmful mutation can never lose the best solution found so far.

Q36. A population has fitnesses $500, 8, 6, 4$. Compute the selection probabilities, diagnose the problem and give three remedies.

Solution. $\sum f = 518$.

$$p_1 = \frac{500}{518} = \mathbf{0.965}, \quad p_2 = 0.0154, \quad p_3 = 0.0116, \quad p_4 = 0.0077.$$

Expected copies of the first individual with $N = 4$: 3.86 — it will fill almost the entire mating

pool.

Diagnosis: premature convergence. Within a generation or two every chromosome will be a near-copy of the first. Crossover between identical parents reproduces the parents, so the search stalls at whatever local optimum that individual represents, long before the global optimum is found.

Remedies.

1. **Tournament selection** — pick k individuals at random and keep the best; pressure then depends on *rank*, not on the fitness magnitudes, so one huge value cannot dominate.
2. **Rank-based selection or fitness scaling** — compress the fitness range before computing p_i.
3. **Increase the mutation rate or the population size** — both restore diversity directly.

(Elitism should be retained alongside these so that raising mutation cannot lose the best solution.)

Q37. Compute the simulated-annealing acceptance probability for $\Delta = +6$ at $T = 12$, $T = 3$ and $T = 1$. *Use* $e^{-0.5} = 0.6065$, $e^{-2} = 0.1353$, $e^{-6} = 0.00248$.

Solution. $P = e^{-\Delta/T}$ for an uphill move.

T	Δ/T	P	Behaviour
12	0.5	**0.6065**	accepts most uphill moves — exploration
3	2.0	**0.1353**	selective
1	6.0	**0.00248**	almost pure hill-climbing — exploitation

As T falls the same uphill move becomes some 245 times less likely to be accepted. This is the mechanism by which annealing hands over smoothly from exploration to exploitation.

Q38. A simulated-annealing run starts at $T_0 = 100$ with cooling factor $\alpha = 0.9$. Find the temperature after 5 steps, and how many steps are needed to fall below $T = 1$.

Solution. $T_k = T_0\alpha^k = 100(0.9)^k$.

$$T_5 = 100(0.9)^5 = 100(0.59049) = \mathbf{59.05}.$$

For $T_k < 1$:

$$100(0.9)^k < 1 \Rightarrow (0.9)^k < 0.01 \Rightarrow k > \frac{\ln 0.01}{\ln 0.9} = \frac{-4.6052}{-0.10536} = 43.7.$$

$k = 44$ **steps.** *Design note:* a slower schedule (larger α) explores longer and finds better solutions but costs more iterations — the standard quality/time trade-off.

Q39. Perform one PSO velocity and position update for a particle with $x = 4$, $v = -2$, $p = 6$, $g = 10$, $w = 0.5$, $c_1 = c_2 = 2$, $r_1 = 0.6$, $r_2 = 0.4$. Identify which term dominates.

Solution.

$$\begin{aligned}
\text{inertia} &= wv = 0.5(-2) = -1.0 \\
\text{cognitive} &= c_1 r_1 (p - x) = 2(0.6)(6 - 4) = 2.4 \\
\text{social} &= c_2 r_2 (g - x) = 2(0.4)(10 - 4) = 4.8 \\
v_{\text{new}} &= -1.0 + 2.4 + 4.8 = \mathbf{6.2} \\
x_{\text{new}} &= 4 + 6.2 = \mathbf{10.2}
\end{aligned}$$

The social term dominates (4.8 of the 6.2 total), because the swarm best lies furthest from the particle. The inertia term is negative — the particle was moving away — but is overwhelmed, so the particle reverses direction.
Note the particle *overshoots* past $g = 10$ to 10.2. This is intentional: it lets PSO explore just beyond the current best rather than merely converging onto it. A velocity clamp $v_{\max}$ is usually applied to stop the swarm exploding.

Q40. An ant at city 1 may move to city 2 or city 3, with $\tau_{12} = 4$, $d_{12} = 5$, $\tau_{13} = 1$, $d_{13} = 2$, $\alpha = 1$, $\beta = 2$. Compute both transition probabilities. Then update τ_{13} given $\rho = 0.4$, $Q = 60$ and an ant tour of length 30 using that edge.

Solution. $\eta = 1/d$: $\eta_{12} = 0.2$, $\eta_{13} = 0.5$.

Edge	τ^α	η^β	Product	Probability
$1 \to 2$	4	$0.2^2 = 0.04$	0.16	$0.16/0.41 = \mathbf{0.390}$
$1 \to 3$	1	$0.5^2 = 0.25$	0.25	$0.25/0.41 = \mathbf{0.610}$
			$\sum = 0.41$	1.000 ✓

City 3 is more likely (61%) *despite having only a quarter of the pheromone*, because $\beta = 2$ weights the much shorter distance heavily.
Pheromone update for edge $1 \to 3$, used by an ant with tour length $L = 30$:

$$\tau_{13} \leftarrow (1-\rho)\tau_{13} + \frac{Q}{L} = 0.6(1) + \frac{60}{30} = 0.6 + 2 = \mathbf{2.6}.$$

The unused edge $1 \to 2$ merely evaporates: $\tau_{12} \leftarrow 0.6(4) = \mathbf{2.4}$. After one iteration the two edges are nearly level — evaporation (ρ) provides exploration, while reinforcement (Q/L, larger for shorter tours) provides exploitation.

Q41. State the No Free Lunch theorem and explain what it implies for choosing between an exact algorithm and a metaheuristic.

Solution. **No Free Lunch (Wolpert & Macready, 1997).** Averaged over *all* possible objective functions, every optimisation algorithm has identical expected performance. No algorithm is universally superior to any other, including random search.
Implication for algorithm choice. Superiority comes only from *matching the algorithm to the structure of the problem at hand* — there is no general-purpose best optimiser. Concretely:

- If the problem has exploitable structure and a known polynomial algorithm (sorting, shortest path, MST, matching, max flow), **use that exact algorithm**. Never use a genetic algorithm to sort a list — it would be slower and would not even guarantee correctness.
- If the problem is NP-hard but small ($n \lesssim 30$) or heavily constrained, use **backtracking or branch and bound**, which still return the exact optimum.
- If it is NP-hard, large, and the objective is cheap to evaluate but lacks structure, use a **metaheuristic** — GA for discrete or permutation problems, PSO or differential evolution for continuous ones, ACO for routing, simulated annealing where a good neighbourhood exists.

Whichever is chosen, report the **gap to a bound** (an LP relaxation, or the MST lower bound for TSP) so that a "good enough" claim is quantified rather than asserted.